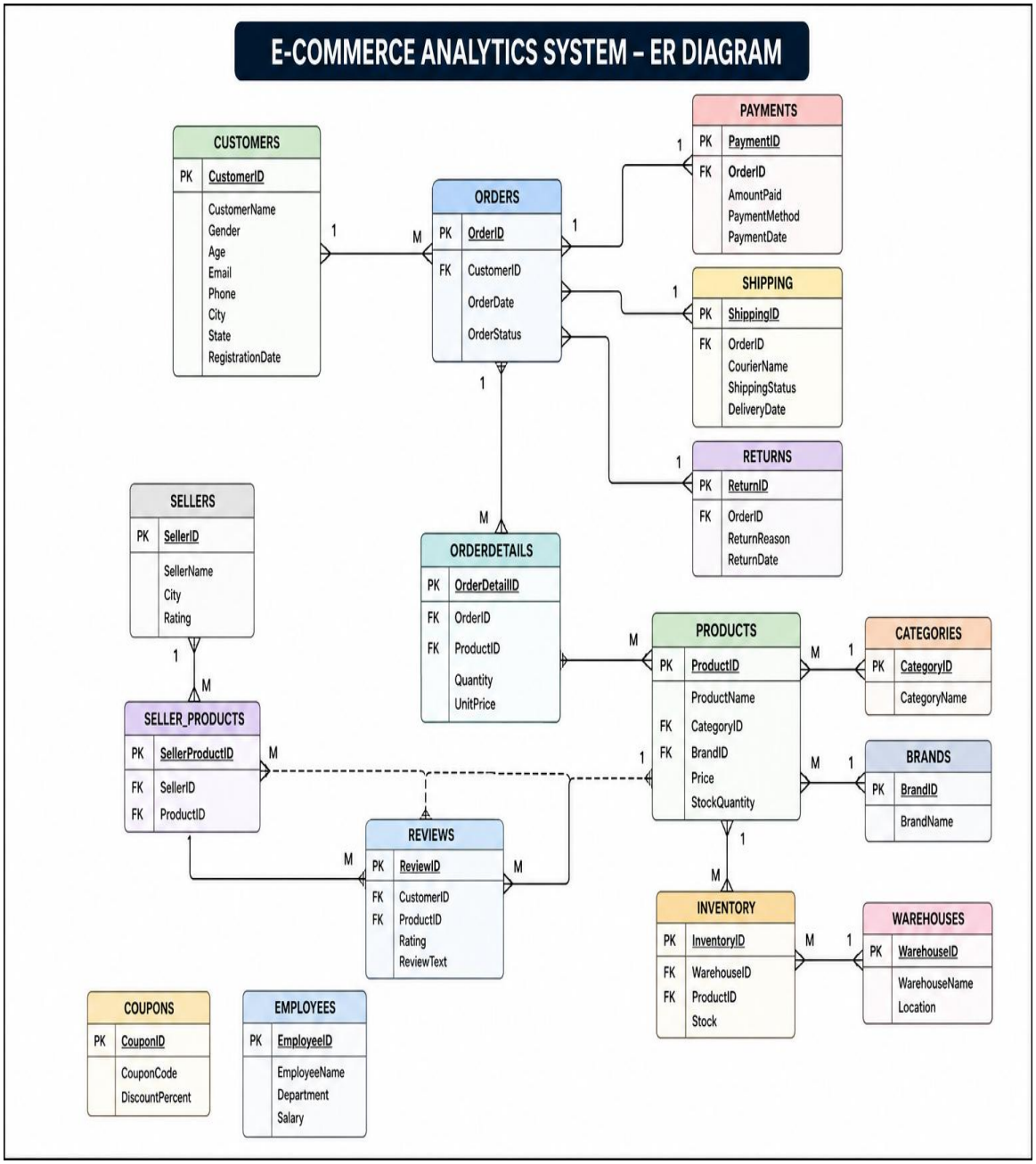
E-COMMERCE ANALYTICS SYSTEM

ID: 148239

Name: Pooja S

GitHub URL: https://github.com/pooja-shindhe/ECAS

Table Schema:



BASIC LEVEL

1. Find customers above age 60.

```
SELECT *
FROM customers
WHERE Age > 60;
```

2. Retrieve all orders placed today.

```
SELECT *
FROM orders
WHERE OrderDate = CURDATE();
```

3. Find total customers available in each city.

```
SELECT City, COUNT(*) TotalCustomers
FROM customers
GROUP BY City;
```

4. Display products with stock quantity below 100.

```
SELECT *
FROM products
WHERE StockQuantity < 100;
```

5. Find total orders placed by each customer.

```
SELECT CustomerID,
COUNT(*) TotalOrders
FROM orders
GROUP BY CustomerID;
```

6. Retrieve customers who never placed an order.

```
SELECT *
FROM customers
WHERE CustomerID NOT IN
(
   SELECT CustomerID
   FROM orders
);
```

7. Find products priced above ₹50,000.

```
SELECT *
FROM products
WHERE Price > 50000;
```

8. Display top 10 highest-priced products.

```
SELECT *
FROM products
ORDER BY Price DESC
LIMIT 10;
```

9. Find total inventory stock warehouse-wise.

```
SELECT WarehouseID,
SUM(Stock) TotalStock
FROM inventory
GROUP BY WarehouseID;
```

10. Find total revenue generated through payments.

```
SELECT SUM(AmountPaid) TotalRevenue
FROM payments;
```

INTERMEDIATE LEVEL

11. Top 5 customers by spending.

```
SELECT c.CustomerID,
c.CustomerName,
SUM(p.AmountPaid) TotalSpent
FROM customers c
JOIN orders o
ON c.CustomerID=o.CustomerID
JOIN payments p
ON o.OrderID=p.OrderID
GROUP BY c.CustomerID,c.CustomerName
ORDER BY TotalSpent DESC
LIMIT 5;
```

12. Top 10 products by sales revenue.

```
SELECT ProductID,
SUM(Quantity*UnitPrice) Revenue
FROM orderdetails
GROUP BY ProductID
ORDER BY Revenue DESC
LIMIT 10;
```

13. Category generating maximum revenue.

```
SELECT c.CategoryName,
SUM(od.Quantity*od.UnitPrice) Revenue
FROM categories c
JOIN products p
ON c.CategoryID=p.CategoryID
JOIN orderdetails od
ON p.ProductID=od.ProductID
GROUP BY c.CategoryName
ORDER BY Revenue DESC
LIMIT 1;
```

14. Most reviewed product.

```
SELECT ProductID,
COUNT(*) ReviewCount
FROM reviews
GROUP BY ProductID
ORDER BY ReviewCount DESC
LIMIT 1;
```

15. Customers ordering from multiple categories.

```
SELECT o.CustomerID
FROM orders o
JOIN orderdetails od
ON o.OrderID=od.OrderID
JOIN products p
ON od.ProductID=p.ProductID
GROUP BY o.CustomerID
HAVING COUNT(DISTINCT p.CategoryID)>1;
```

16. Top 10 cities generating maximum revenue.

```
SELECT c.City,
SUM(p.AmountPaid) Revenue
FROM customers c
JOIN orders o
ON c.CustomerID=o.CustomerID
JOIN payments p
ON o.OrderID=p.OrderID
GROUP BY c.City
ORDER BY Revenue DESC
LIMIT 10;
```

17. Products never sold.

```
SELECT p.*
FROM products p
LEFT JOIN orderdetails od
ON p.ProductID=od.ProductID
WHERE od.ProductID IS NULL;
```

18. Most frequently used payment method.

```
SELECT PaymentMethod,
COUNT(*) UsageCount
FROM payments
GROUP BY PaymentMethod
ORDER BY UsageCount DESC
LIMIT 1;
```

19. Repeat customers.

```
SELECT CustomerID,
COUNT(*) OrderCount
FROM orders
GROUP BY CustomerID
HAVING COUNT(*)>1;
```

20. Monthly sales trend.

```
SELECT YEAR(PaymentDate),
MONTH(PaymentDate),
SUM(AmountPaid)
FROM payments
GROUP BY YEAR(PaymentDate),
MONTH(PaymentDate);
```

ADVANCED SQL (STORED PROCEDURE, TRIGGERS)

21. Find the Second Highest Spending Customer

```
SELECT *
FROM
(
   SELECT
      c.CustomerID,
      c.CustomerName,
      SUM(p.AmountPaid) TotalSpent,
      DENSE_RANK() OVER(
         ORDER BY SUM(p.AmountPaid) DESC
      ) AS rn
   FROM customers c
   JOIN orders o
      ON c.CustomerID=o.CustomerID
   JOIN payments p
      ON o.OrderID=p.OrderID
   GROUP BY c.CustomerID,c.CustomerName
) x
WHERE rn=2;
```

22. Product Revenue Ranking

```
SELECT
   ProductID,
   SUM(Quantity*UnitPrice) Revenue,
   RANK() OVER(
      ORDER BY SUM(Quantity*UnitPrice) DESC
   ) ProductRank
FROM orderdetails
GROUP BY ProductID;
```

23. Customer Lifetime Value (CLV)

```
SELECT
   c.CustomerID,
   c.CustomerName,
   SUM(p.AmountPaid) LifetimeValue
FROM customers c
JOIN orders o
   ON c.CustomerID=o.CustomerID
JOIN payments p
```

```
   ON o.OrderID=p.OrderID
GROUP BY c.CustomerID,c.CustomerName
ORDER BY LifetimeValue DESC;
```

24. Detect Suspicious Payments

```
SELECT
   o.OrderID,
   SUM(od.Quantity*od.UnitPrice) OrderValue,
   SUM(p.AmountPaid) PaidValue
FROM orders o
JOIN orderdetails od
   ON o.OrderID=od.OrderID
JOIN payments p
   ON o.OrderID=p.OrderID
GROUP BY o.OrderID
HAVING PaidValue > OrderValue;
```

25. Top 10 Customers using CTE

```
WITH CustomerRevenue AS
(
   SELECT
      c.CustomerID,
      c.CustomerName,
      SUM(p.AmountPaid) Revenue
   FROM customers c
   JOIN orders o
      ON c.CustomerID=o.CustomerID
   JOIN payments p
      ON o.OrderID=p.OrderID
   GROUP BY c.CustomerID,c.CustomerName
)
SELECT *
FROM CustomerRevenue
ORDER BY Revenue DESC
LIMIT 10;
```

26. Running Revenue Month-over-Month

```
SELECT
   YEAR(PaymentDate) YearNo,
   MONTH(PaymentDate) MonthNo,
   SUM(AmountPaid) Revenue,
```

```
   SUM(SUM(AmountPaid))
   OVER(
      ORDER BY YEAR(PaymentDate),
      MONTH(PaymentDate)
   ) RunningRevenue
FROM payments
GROUP BY
   YEAR(PaymentDate),
   MONTH(PaymentDate);
```

27. Category Revenue Contribution %

```
SELECT
   c.CategoryName,
   ROUND(
      SUM(od.Quantity*od.UnitPrice)*100/
      (
         SELECT SUM(Quantity*UnitPrice)
         FROM orderdetails
      ),
      2
   ) ContributionPercent
FROM categories c
JOIN products p
   ON c.CategoryID=p.CategoryID
JOIN orderdetails od
   ON p.ProductID=od.ProductID
GROUP BY c.CategoryName;
```

28. Customers Spending Above Average

```
SELECT *
FROM
(
   SELECT
      c.CustomerID,
      c.CustomerName,
      SUM(p.AmountPaid) TotalSpent
   FROM customers c
   JOIN orders o
      ON c.CustomerID=o.CustomerID
   JOIN payments p
      ON o.OrderID=p.OrderID
   GROUP BY c.CustomerID,c.CustomerName
) x
WHERE TotalSpent >
(
```

```
  SELECT AVG(CustomerSpend)
  FROM
  (
     SELECT
        SUM(p.AmountPaid) CustomerSpend
     FROM customers c
     JOIN orders o
        ON c.CustomerID=o.CustomerID
     JOIN payments p
        ON o.OrderID=p.OrderID
     GROUP BY c.CustomerID
  ) y
);
```

29. Customers Spending Above Their City's Average

```
 SELECT
    c.CustomerID,
    c.CustomerName,
    c.City,
    SUM(p.AmountPaid) TotalSpent
 FROM customers c
 JOIN orders o
    ON c.CustomerID = o.CustomerID
 JOIN payments p
    ON o.OrderID = p.OrderID
 GROUP BY
    c.CustomerID,
    c.CustomerName,
    c.City
 HAVING SUM(p.AmountPaid) >
 (
    SELECT AVG(CustomerRevenue)
    FROM
    (
       SELECT
          c2.CustomerID,
          SUM(p2.AmountPaid) CustomerRevenue
       FROM customers c2
       JOIN orders o2
          ON c2.CustomerID = o2.CustomerID
       JOIN payments p2
          ON o2.OrderID = p2.OrderID
       WHERE c2.City = c.City
       GROUP BY c2.CustomerID
    ) CityCustomers
 );
```

30. Find Customers Who Have Placed Orders

```
SELECT *
FROM customers c
WHERE EXISTS
(
   SELECT 1
   FROM orders o
   WHERE o.CustomerID=c.CustomerID
);
```

31. Find Customers Who Never Placed Orders

```
SELECT *
FROM customers c
WHERE NOT EXISTS
(
   SELECT 1
   FROM orders o
   WHERE o.CustomerID=c.CustomerID
);
```

32. Frequently Purchased Together Products

```
SELECT
   od1.ProductID ProductA,
   od2.ProductID ProductB,
   COUNT(*) Frequency
FROM orderdetails od1
JOIN orderdetails od2
ON od1.OrderID=od2.OrderID
AND od1.ProductID < od2.ProductID
GROUP BY
od1.ProductID,
od2.ProductID;
```

33. Rank Products Within Each Category

```
SELECT
   p.CategoryID,
   p.ProductID,
   SUM(od.Quantity*od.UnitPrice) Revenue,
   RANK() OVER
   (
      PARTITION BY p.CategoryID
      ORDER BY SUM(od.Quantity*od.UnitPrice) DESC
   ) CategoryRank
FROM products p
JOIN orderdetails od
ON p.ProductID=od.ProductID
GROUP BY
p.CategoryID,
p.ProductID;
```

34. Month-over-Month Growth %

```
WITH MonthlySales AS
(
   SELECT
      YEAR(PaymentDate) AS Yr,
      MONTH(PaymentDate) AS Mn,
      SUM(AmountPaid) AS Revenue
   FROM payments
   GROUP BY
      YEAR(PaymentDate),
      MONTH(PaymentDate)
)
SELECT
   Yr,
   Mn,
   Revenue,
   LAG(Revenue) OVER(ORDER BY Yr, Mn) AS PreviousMonthRevenue,
   ROUND(
      (
         Revenue -
         LAG(Revenue) OVER(ORDER BY Yr, Mn)
      ) * 100.0 /
      LAG(Revenue) OVER(ORDER BY Yr, Mn),
      2
   ) AS GrowthPercent
FROM MonthlySales;
```

35. Customer Segmentation

```
CASE
WHEN Revenue > 100000 THEN 'Platinum'
```

```
WHEN Revenue > 50000 THEN 'Gold'
WHEN Revenue > 20000 THEN 'Silver'
ELSE 'Bronze'
END
```

36. Create Customer Revenue View

```
CREATE VIEW vw_customer_revenue AS
SELECT
c.CustomerID,
c.CustomerName,
SUM(p.AmountPaid) Revenue
FROM customers c
JOIN orders o
ON c.CustomerID=o.CustomerID
JOIN payments p
ON o.OrderID=p.OrderID
GROUP BY
c.CustomerID,
c.CustomerName;
```

37. Create Product Sales View

```
CREATE VIEW vw_product_sales AS
SELECT
p.ProductID,
p.ProductName,
SUM(od.Quantity*od.UnitPrice) Revenue
FROM products p
JOIN orderdetails od
ON p.ProductID=od.ProductID
GROUP BY
p.ProductID,
p.ProductName;
```

38. Get Customer Orders by Customer ID

```
DROP PROCEDURE IF EXISTS GetCustomerOrders;

DELIMITER //

CREATE PROCEDURE GetCustomerOrders
(
   IN p_customerid INT
)
BEGIN
```

```
    SELECT
        OrderID,
        CustomerID,
        OrderDate,
        OrderStatus
    FROM orders
    WHERE CustomerID = p_customerid;

END //

DELIMITER ;

CALL GetCustomerOrders(101);
```

39. Monthly Sales Report

```
DROP PROCEDURE IF EXISTS MonthlySalesReport;

DELIMITER //

CREATE PROCEDURE MonthlySalesReport()
BEGIN

    SELECT
        YEAR(PaymentDate) AS YearNo,
        MONTH(PaymentDate) AS MonthNo,
        SUM(AmountPaid) AS Revenue
    FROM payments
    GROUP BY
        YEAR(PaymentDate),
        MONTH(PaymentDate)
    ORDER BY
        YearNo,
        MonthNo;

END //

DELIMITER ;


CALL MonthlySalesReport();
```

40.Top Customer Report

```
DROP PROCEDURE IF EXISTS TopCustomers;
```

```
DELIMITER //

CREATE PROCEDURE TopCustomers()
BEGIN

   SELECT
      c.CustomerID,
      c.CustomerName,
      SUM(p.AmountPaid) AS Revenue
   FROM customers c
   JOIN orders o
      ON c.CustomerID = o.CustomerID
   JOIN payments p
      ON o.OrderID = p.OrderID
   GROUP BY
      c.CustomerID,
      c.CustomerName
   ORDER BY Revenue DESC
   LIMIT 10;

END //

DELIMITER ;


CALL TopCustomers();
```